Configuración del Entorno de Desarrollo Integrado (IDE) y Primer Programa Ejecutable en C++ Proyecto: (Hola Mundo) — LPR 2026

Nombre Completo del Estudiante: Murray Rothbard (Grupo 03)
Nombre de la Institución: E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
Materia: Laboratorio de Programación (LPR) — 5° Año
Profesor: Prof. Mansilla Muñoz York Elías
Estudiantes: Argento Coqui y Lionel Andrés Messi Cuccittini
Fecha: 31 de Agosto de 2026

Introducción

En el desarrollo profesional de software, la productividad de un técnico depende directamente de la correcta elección, preparación y domino de su entorno de trabajo (*Tooling*). A diferencia de lenguajes interpretados o de alto nivel con entornos abstraídos, el lenguaje C++ exige comprender la cadena completa de construcción (*Build Toolchain*): la edición del código fuente en texto plano, la invocación explícita del compilador nativo (como $g++$) y la ejecución del archivo binario directamente en la interfaz de línea de comandos.

El propósito de este trabajo es documentar la instalación y preparación del entorno de desarrollo integrado Visual Studio Code (VS Code), junto con el compilador MinGW-w64 (GCC), verificando la compilación exitosa de un programa ejecutable de consola (src/main.cpp). Asimismo, se detalla la reestructuración del repositorio Git aplicando convenciones de nombres estrictamente en minúsculas para garantizar la compatibilidad entre distintos sistemas operativos.

Desarrollo: Cuestionario Técnico de Control

A continuación se presentan las respuestas fundamentadas al cuestionario de control para aspirantes a Técnico en Programación:

1. ¿Qué entorno de desarrollo te parece más intuitivo a primera vista? ¿Por qué?

Visual Studio Code resulta el entorno más intuitivo a primera vista debido a su interfaz minimalista, ligera y modular. A diferencia de IDEs monolíticos pesados que abruman con barras de herramientas complejas, VS Code presenta un panel lateral limpio y centrado en la edición de texto, permitiendo incorporar herramientas avanzadas (como resaltado de sintaxis y depuración) de forma gradual mediante extensiones.

2. ¿Qué características de un IDE creés que te facilitan más el aprendizaje?

Las características fundamentales que aceleran el aprendizaje son:

- **Resaltado de sintaxis (*Syntax Highlighting*):** Permite diferenciar visualmente palabras clave, variables, cadenas de texto y comentarios.
- **Autocompletado inteligente (*IntelliSense*):** Sugiere funciones y muestra la firma exacta de parámetros sin necesidad de memorizar bibliotecas estándar completas.
- **Detección de errores en tiempo real (*Linter*):** Señala errores sintácticos (como la falta de un punto y coma ;) antes de invocar la compilación.
- **Terminal integrada:** Facilita ejecutar comandos de compilación directamente en el espacio de trabajo.

3. De las tareas realizadas, ¿cuál te resultó más difícil de configurar?

La tarea de mayor complejidad técnica fue la configuración de las variables de entorno del sistema (*System Path*) para el compilador MinGW-w64 ($g++$). Vincular correctamente la ruta del binario en la consola de Windows para que el comando $g++$ sea reconocido de forma global en PowerShell requiere atención meticulosa a los permisos y variables de sistema.

4. ¿Qué diferencia técnica encontrás entre un "Editor de Texto" y un "IDE"?

- **Editor de Texto (ej. Sublime Text / Notepad++):** Es un programa diseñado exclusivamente para manipular caracteres planos. No posee internamente la capacidad de traducir código a lenguaje máquina ni de depurar procesos.
- **IDE / Entorno de Desarrollo Integrado (ej. Visual Studio Community):** Es una suite completa que integra en una misma aplicación un editor de código, un

compilador/enlazador (*Linker*), un depurador (*Debugger*) paso a paso y gestores de construcción de proyectos.

5. ¿Qué tipo de aplicaciones sueñan con desarrollar al recibirse en 6° y 7° año? ¿Por qué?

El objetivo principal es orientar la formación hacia el desarrollo de **motores de videojuegos 2D/3D y sistemas de backend de alta performance**. C++ es el estándar indiscutido de la industria para aplicaciones que exigen control estricto de memoria RAM, baja latencia y alta tasa de refresco por segundo.

6. ¿Qué otros lenguajes de programación te gustaría investigar además de C y C++?

Resulta estratégico investigar **C# (C-Sharp)** por su integración nativa con el motor Unity para videojuegos y aplicaciones empresariales .NET, y **Python** por su relevancia mundial en automatización de scripts, analítica de datos e Inteligencia Artificial.

7. ¿Qué tan importante considerás que es el orden y la nomenclatura de tus archivos y carpetas?

El orden y la nomenclatura uniforme son críticos en la industria del software. Utilizar nombres claros, sin espacios, sin tildes y estrictamente en **minúsculas** (convención *kebab-case* o *snake_case*) previene errores de resolución de rutas (*pathing*) cuando el repositorio es clonado en sistemas operativos con distinta sensibilidad a mayúsculas (*Case-Sensitivity*) como Linux o macOS.

8. Si pudieras diseñar el "entorno ideal", ¿qué herramienta o funcionalidad le agregarías?

El entorno ideal incorporaría un analizador visual de memoria RAM en tiempo real que muestre en una ventana gráfica el comportamiento exacto de la Pila (*Stack*) y el Montículo (*Heap*) a medida que el ejecutable ejecuta cada línea de código.

9. ¿Cómo creés que la Inteligencia Artificial puede ayudar a un programador sin que este pierda su capacidad de razonamiento?

La Inteligencia Artificial debe utilizarse como un tutor o asistente de revisión de código (*Code Reviewer*), solicitándole explicaciones teóricas sobre errores de compilación o sugerencias de optimización. No debe emplearse para generar código a ciegas, garantizando que el estudiante comprenda la lógica algorítmica subyacente.

10. ¿Cuál es tu meta principal para este año en la materia LPR?

La meta principal es dominar los fundamentos de la programación estructurada y la Programación Orientada a Objetos en C++, aprendiendo a gestionar repositorios profesionales en Git y redactar documentación técnica bajo estándares industriales.

Estructura Normalizada del Repositorio (holamundo)

Respetando la norma de minúsculas para todos los directorios del proyecto, la estructura oficial queda definida de la siguiente manera:

```
/holamundo/
├── .gitignore            <-- Archivo para que Git ignore carpetas de configuración (.vscode)
├── README.md      <-- Archivo explicativo del entorno que configuraron
├── LICENSE           <-- Descripción breve de la licencia
├── docs/                 <-- Carpeta exclusiva para documentación
│   └── InformeProyectoholamundo.pdf        <-- ENTREGABLE (En PDF con formato APA v7)
├── src/                        <-- Carpeta para los códigos fuente
│   └── main.cpp                  <-- Primer código de prueba en C++
└── capturas/                   <-- Carpeta para guardar las evidencias (Imágenes .png)
    ├── gpp_version.png           <-- Captura de la consola verificando el compilador
    ├── extensiones_vscode.png      <-- Captura de las extensiones instaladas
    └── ejecucion_hola_mundo.png    <-- Captura de la terminal compilando y corriendo el
programa
```

Conclusión

La configuración exitosa de VS Code junto con la cadena de herramientas MinGW ($g\ ++$) sienta las bases operativas para abordar el ciclo lectivo de Laboratorio de Programación. El aprendizaje de los comandos de compilación explícitos desde la terminal PowerShell (g++ src/main.cpp -o src/holamundo.exe) afianza la comprensión sobre cómo el código fuente se transforma en un ejecutable binario. La estandarización de nombres de directorios en minúsculas consolida las buenas prácticas de versionado en Git.

Referencias

- Kernighan, B. W., & Ritchie, D. M. (1988). *El lenguaje de programación C* (2.ª ed.). Prentice Hall.
- Mansilla Muñoz, Y. E. (2026). *Guía docente: Actividad 2 — El taller del programador*. Escuela de Educación Secundaria Técnica N° 1 "Eduardo Ader".
- Microsoft. (2026). *C++ programming with Visual Studio Code*. https://code.visualstudio.com/docs/cpp/config-mingw